

3-Bit Counter

Implementation using Xilinx Vivado

Design Report

By Ahmed H. Ashour

Introduction

To implement a 3-bit counter driven by a 1 Hz clock signal. The design uses the xc7z020clg484-2 Part's internal oscillator that goes into a PLL to generate a 8 MHz signal then it is then divided down to 1 Hz using a clock divider.

Counter code

```
module counter(  
    input clk,  
    input rst_n,  
    output reg [2:0] counter  
);  
  
always @ (posedge clk or negedge rst_n) begin  
    if(!rst_n) begin  
        counter <= 3'b000;  
    end else begin  
        counter <= counter + 1'b1;  
    end  
end  
endmodule
```

Figure 1: Counter code

Clock Divider Code

```
module clk_divider (  
    input clk_in,  
    input rst_n,  
    output reg clk_out  
);  
    //divide from 8mhz to 1hz  
    reg [21:0] counter; // 22 bits to count 4000000  
    always @(posedge clk_in or negedge rst_n) begin  
        if (!rst_n) begin  
            counter <= 0;  
            clk_out <= 0;  
        end else begin  
            if (counter == 3_999_999) begin  
                counter <= 0;  
                clk_out <= ~clk_out;  
            end else begin  
                counter <= counter + 1;  
            end  
        end  
    end  
endmodule
```

Figure 2 : Clock Divider code

Instantiating the PLL IP into the top module

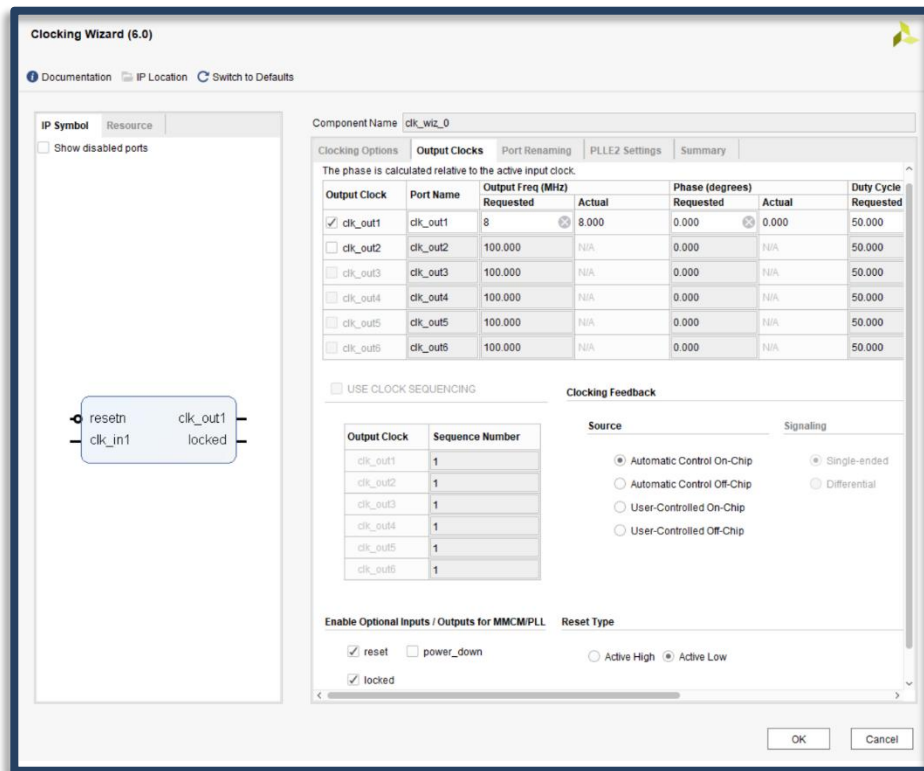


Figure 3 : PLL IP

Top Module

```

module counter_top(
    input clk,
    input rst_n,
    output [2:0] counter);

    wire clk_pll, locked_pll;
    wire clk_div;
    //----- Begin Cut here for INSTANTIATION Template ---// INST_TAG

    clk_wiz_0_pll
    (
        // Clock out ports
        .clk_out1(clk_pll), // output clk_out1
        // Status and control signals
        .resetn(rst_n), // input resetn
        .locked(locked_pll), // output locked
        // Clock in ports
        .clk_in1(clk): // input clk_in1
    );
    // INST_TAG_END ----- End INSTANTIATION Template -----

    clk_divider div1(
        .clk_in(clk_pll),
        .clk_out(clk_div),
        .rst_n(locked_pll));

    counter cnt1 (
        .clk(clk_div),
        .rst_n(locked_pll),
        .counter(counter));
endmodule
    
```

Figure 4 : Top module code

Behavioral simulation

For the simulation the divider was set to output 1MHz clk_out since simulating a 1Hz signal is not practical

```
timescale 1ns/1ps

module counter_tb:

    reg clk;
    reg rst_n;
    wire [2:0] counter;

    counter_top dut (
        .clk (clk),
        .rst_n (rst_n),
        .counter(counter)
    );

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $display("Time\trst_n\tCounter");
        $monitor("%0t\t%0t\t%d", $time, rst_n, counter);

        rst_n = 0;
        #10;
        rst_n = 1;

        #20_000;

        $display("Simulation finished at time %0t", $time);
        $stop;
    end

    initial begin
        $dumpfile("counter_tb.vcd");
        $dumpvars(0, counter_tb);
    end

endmodule
```

Figure 5 : Testbench Code

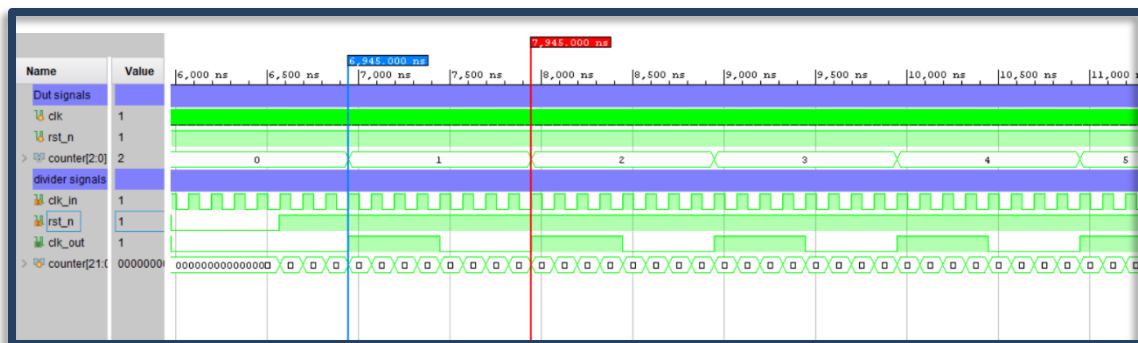


Figure 6 : Simulation waveform

The output waveform show that the divider can output a 1MHz signal correctly

Constraints file for xc7z020clg484-2 part

```
# -----  
# User LEDs - Bank 33  
# -----  
set_property PACKAGE_PIN T22 [get_ports {counter[0]}]  
set_property PACKAGE_PIN T21 [get_ports {counter[1]}]  
set_property PACKAGE_PIN U22 [get_ports {counter[2]}]  
##  
## User DIP Switches - Bank 35  
##  
set_property PACKAGE_PIN F22 [get_ports rst_n]  
##  
# -----  
# Clock Source - Bank 13  
# -----  
set_property PACKAGE_PIN Y9 [get_ports clk]  
# -----  
# Timing cons  
# -----  
create_clock -name clk -period 10 [get_ports clk]  
#clock from PLL output  
create_generated_clock -name counter_clk \  
-source clk \  
-divide_by [expr 100/8] \  
pll/clk_out1  
  
set_false_path -from rst_n -to counter_reg[0]/CLR  
set_false_path -from rst_n -to counter_reg[1]/CLR  
set_false_path -from rst_n -to counter_reg[2]/CLR  
  
set_false_path -from counter_reg[0]/C -to counter[0]  
set_false_path -from counter_reg[1]/C -to counter[1]  
set_false_path -from counter_reg[2]/C -to counter[2]  
  
# Note that the bank voltage for IO Bank 33 is fixed to 3.3V on ZedBoard.  
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 33]]  
  
# Set the bank voltage for IO Bank 34 to 1.8V by default.  
# set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 34]];  
# set_property IOSTANDARD LVCMOS25 [get_ports -of_objects [get_iobanks 34]];  
# set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 34]];  
  
# Set the bank voltage for IO Bank 35 to 1.8V by default.  
# set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 35]];  
# set_property IOSTANDARD LVCMOS25 [get_ports -of_objects [get_iobanks 35]];  
# set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 35]];  
  
# Note that the bank voltage for IO Bank 13 is fixed to 3.3V on ZedBoard.  
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 13]]
```

Figure 7 : constraints file

Elaborated Design Schematic

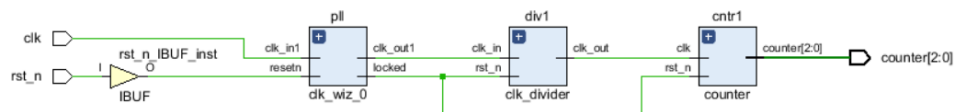


Figure 8 : Elaborated Design Schematic

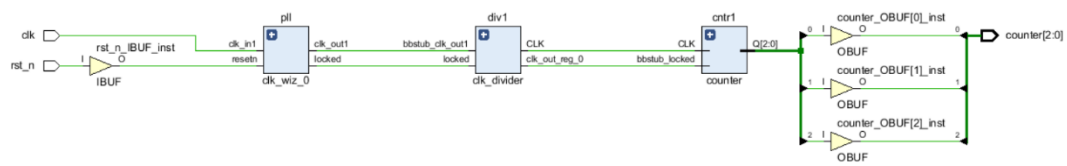


Figure 9 : Synthesized design

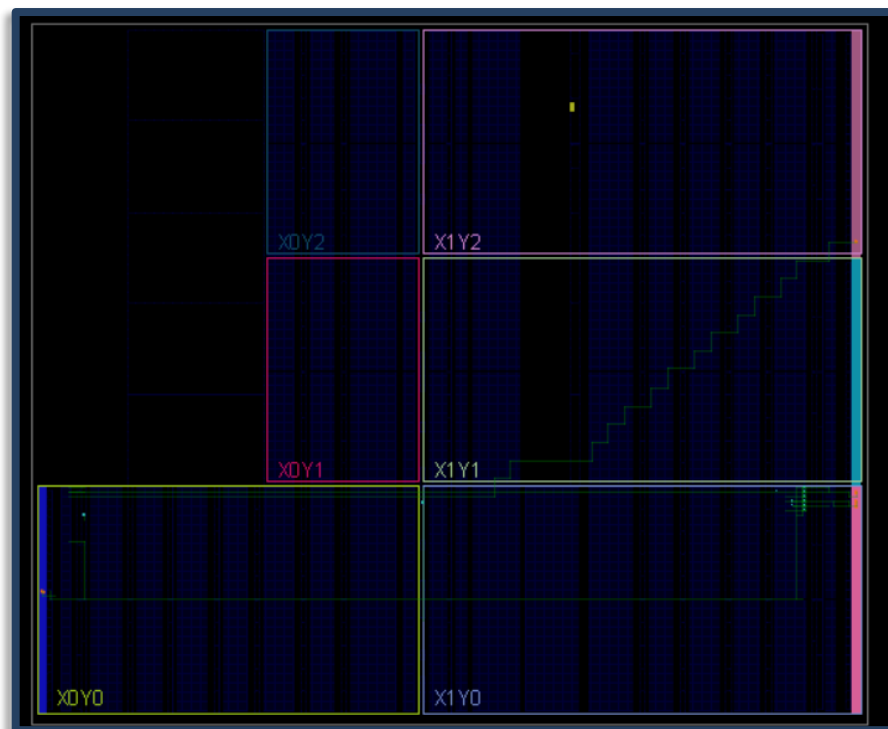


Figure 10 : Implemented Design

Project Summary

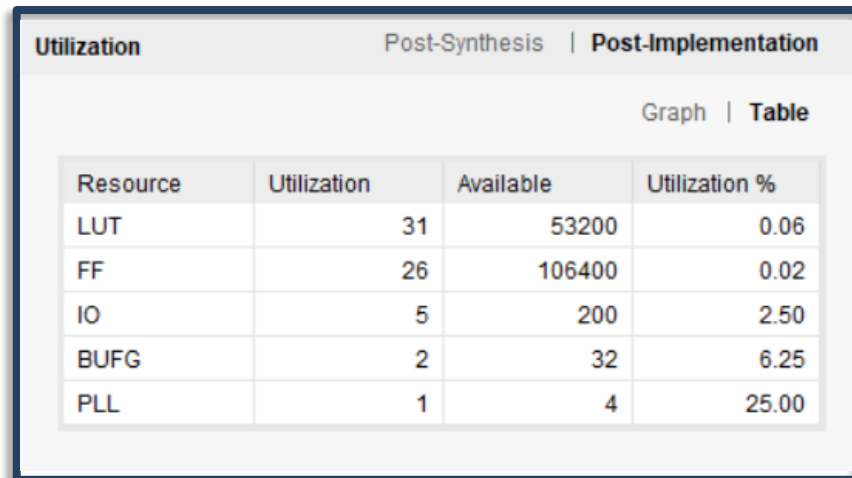


Figure 11 : Utilization



Figure 12 : Timing

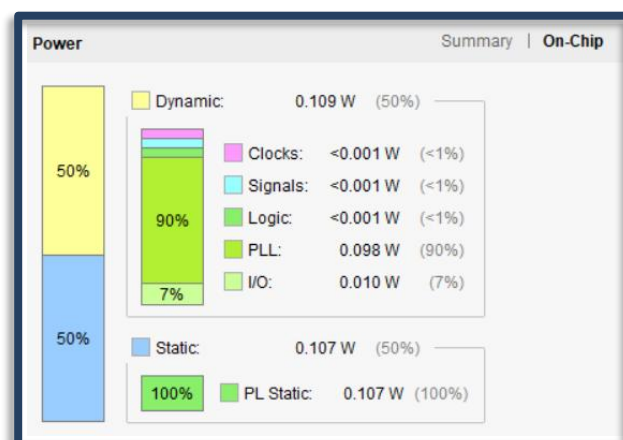


Figure 13 : Power